MCU TINKERING LAB

balancebot

Two-Wheel Self-Balancing Robot

Assembly, Wiring & Firmware Build Guide

A hands-on guide to building the balancebot on a **Seeed Studio XIAO RP2350**: an MPU6050 IMU feeds a 500 Hz balance loop, and the RP2350's PIO state machines step two DRV8825-driven NEMA17 motors. Tuning happens live over a USB serial console, and tuned gains persist to flash.

Target MCU	RP2350 (XIAO, 2 MB flash)
Toolchain	Raspberry Pi Pico SDK 2.2.0 (containerized)
Difficulty	Intermediate

Pin data auto-generated from `src/pin_config.h` via `just balancebot::gen-pin-defs`.
All components must share a common ground.

Contents

1 · Overview . . . 3
2 · Bill of Materials . . . 3
Tools required . . . 4
3 · System Architecture . . . 4
4 · Wiring Reference . . . 5
4.1 · XIAO RP2350 connections . . . 5
4.2 · DRV8825 carriers (each) . . . 5
4.3 · MPU6050 (GY-521) . . . 6
4.4 · XIAO pinout . . . 6
5 · Power . . . 6
6 · Assembly Steps . . . 7
Build the 3S pack . . . 7
Wire it up . . . 7
6.1 · Set the DRV8825 current limit (Vref) . . . 7
7 · Build & Flash the Firmware . . . 8
8 · First Boot & Calibration . . . 8
9 · Functional Checkout . . . 9
10 · Tuning: PID and LQR . . . 9
10.1 · PID . . . 9
10.2 · LQR . . . 9
11 · Troubleshooting . . . 10

1 · Overview

The balancebot stands on two stepper-driven wheels and keeps itself upright by driving the wheels under its own fall. It is the repo's first non-ESP32 project: plain CMake against the Raspberry Pi Pico SDK, built in a container, with no sdkconfig and no `idf.py`.

The two RP2350 cores split the work by deadline. Core 1 owns everything hard real-time; core 0 owns everything a human talks to. Core 0 sends commands to core 1 through a lock-free queue, and core 1 publishes its state back through a seqlock snapshot, so the control loop never blocks and never prints.

Core 1 — 500 Hz control loop
MPU6050 burst read → complementary filter → angle PID or LQR, whose output is wheel **acceleration** → integrated to a step rate → PIO0 step pulses. Tilt cutoff, watchdog and flash saves live here too.

Core 0 — housekeeping
USB CDC command line, CSV telemetry, status LED, battery reading and the LQR gain solve. PIO0 SM0/SM1 generate the STEP pulse trains (30–20 000 steps/s) independently of either core.

Safety model

- A 10 kΩ pull-up holds the DRV8825s' shared nENABLE high, so the drivers are **disabled** while the XIAO boots, resets or sits in BOOTSEL. Firmware also drives nENABLE high as its very first action.
- Tilt cutoff: |pitch| > 45° disables the drivers (FAULT); it clears after 2 s upright within 5°.
- A 100 ms hardware watchdog, fed only by the control loop and enabled at first arm, reboots a wedged loop into motors-off.
- The motors arm only on the `arm` command or a deliberate gesture: hold the robot upright (within 1°) for 3 s.

2 · Bill of Materials

Qty	Component	Notes
1	Seeed XIAO RP2350	Dual Cortex-M33 @ 150 MHz, 2 MB flash, USB-C, no radio
1	MPU6050 on a GY-521 breakout	I²C address 0x68, data-ready INT
2	DRV8825 stepper carrier	STEP/DIR, strapped for 1/8 microstep
2	NEMA17 bipolar stepper	200 steps/rev (1600 at 1/8 µstep)
2	Wheel	~80–100 mm, directly on the motor shafts
3	18650 Li-ion cell	Same model, matched voltage; built into a 3S pack (3S2P with 6 for longer runs)
1	3S BMS board	≥ 10 A, balance taps, over-discharge cutoff
—	Pure nickel strip	0.15 × 8 mm, spot-welded; not nickel-plated steel
1	LM2596 buck module	Battery → 5.0 V → XIAO 5V pad
2	100 µF electrolytic capacitor	Across each DRV8825's VMOT/GND, close to the pins
1	10 kΩ resistor	nENABLE pull-up to 3V3
—	Frame, wiring, headers	Two-deck chassis; IMU mounted rigidly at axle height

Tools required

Soldering iron and solder, a battery spot welder, wire strippers, a small screwdriver for the DRV8825 trim pots, a multimeter (for the buck output, continuity and Vref), a USB-C cable, and a computer with Docker for the containerized build. `picotool` (`brew install picotool`) is optional; without it, flash by drag-and-drop.

> **The XIAO's battery pads are not a motor supply**
> The XIAO RP2350 has a LiPo charger on its battery pads, rated around 370 mA. The motors run from the pack through the DRV8825 VMOT pins; the XIAO takes its power from the buck converter on its **5V** pad. Never wire the motor pack to the XIAO's battery pads.

3 · System Architecture

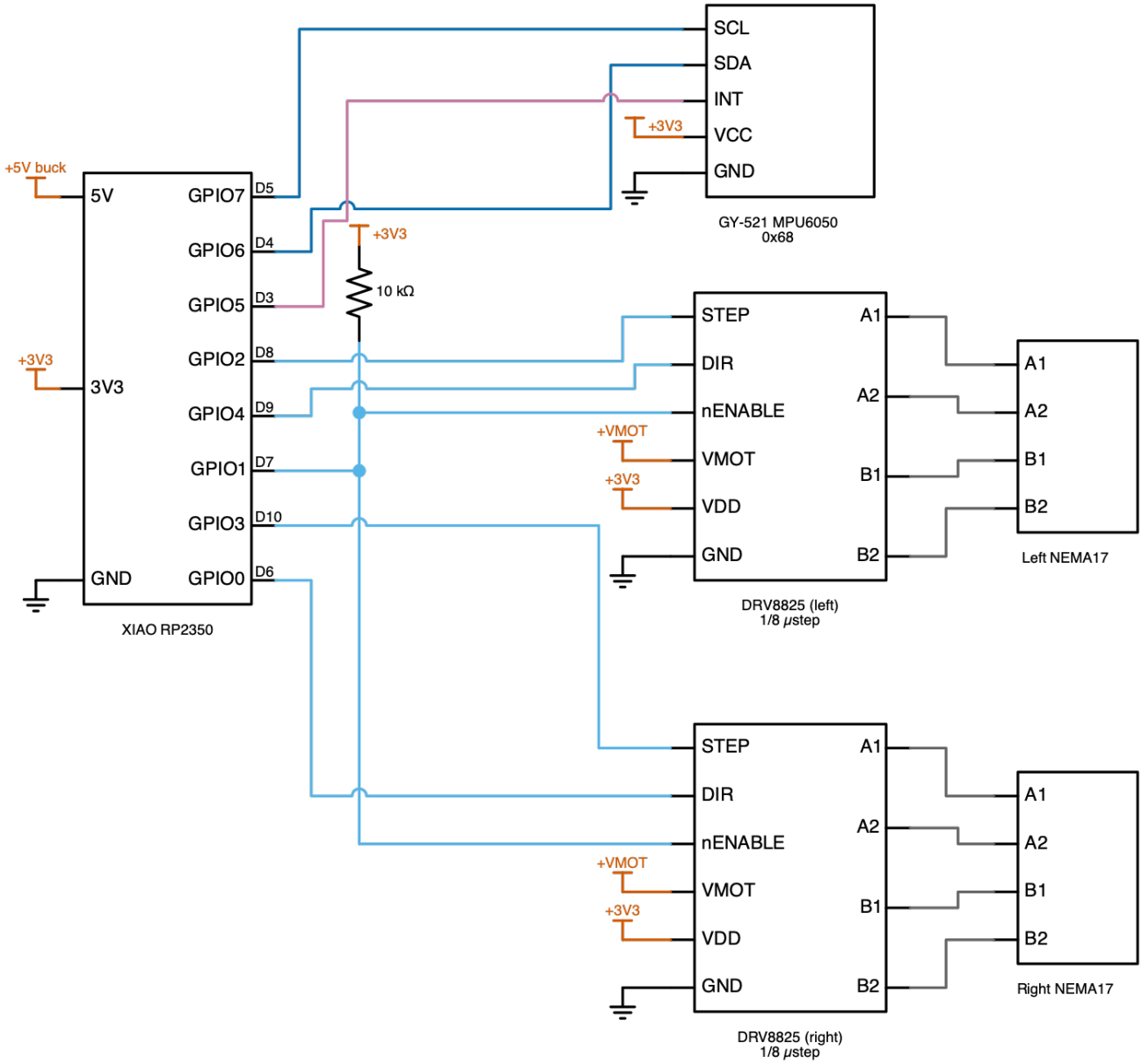


Figure 1: Full wiring schematic. Source: `docs/schematics/circuits/balancebot.py`.

The XIAO drives three groups of signals and nothing else. I²C1 (GPIO6/7) and a data-ready interrupt (GPIO5) go to the MPU6050. A STEP and a DIR line go to each DRV8825 (left GPIO2/4, right GPIO3/0). One shared, active-low nENABLE line (GPIO1) switches both drivers at once.

The angle PID commands wheel **acceleration**, not speed: the loop integrates it into a step rate, and `stepper_set_rate()` owns the acceleration slew and the direction-reversal rules. A slower velocity loop (100 Hz) adds a small lean offset so the robot holds station instead of wandering. The right motor is wired as a mirror image of the left, so firmware inverts its DIR line.

4 · Wiring Reference

4.1 · XIAO RP2350 connections

XIAO pad	GPIO	Connects to	Notes
D3	GPIO5	MPU6050 INT	Data-ready, active high, push-pull
D4	GPIO6	**MPU6050 SDA**	I²C1, 400 kHz
D5	GPIO7	**MPU6050 SCL**	I²C1
D6	GPIO0	Right DRV8825 DIR	Inverted in firmware (mirror-image motor)
D7	GPIO1	Both DRV8825 nENABLE	Active low · **10 kΩ pull-up to 3V3**
D8	GPIO2	Left DRV8825 STEP	PIO0 SM0
D9	GPIO4	Left DRV8825 DIR	
D10	GPIO3	Right DRV8825 STEP	PIO0 SM1
3V3	—	MPU6050 VCC, DRV8825 straps, pull-up	See §4.2
5V	—	Buck converter output	Not the battery pads
GND	—	Common ground	
D0–D2	—	Spare	ADC-capable

Three pins never leave the board: the user LED (GPIO25, active low), the battery-sense ADC (GPIO29, behind a ÷2 divider) and the divider's enable (GPIO19). `WIRING.md` carries the same table generated from `hardware.toml`.

4.2 · DRV8825 carriers (each)

DRV8825 pin	Connect to
VMOT / GND MOT	Battery + / − with a 100 µF electrolytic across, close to the pins
B2 B1 A1 A2	Stepper coils, one coil pair per letter. Swap one pair to reverse a motor in hardware
GND LOGIC	Common ground
STEP / DIR	XIAO, per §4.1
nENABLE	The shared GPIO1 line, with its 10 kΩ pull-up to 3V3
nSLEEP + nRESET	Both tied to 3V3 (or bridge nSLEEP–nFAULT on carriers with that route)
M0 M1 M2	1/8 microstep strap: M0 = HIGH, M1 = HIGH, M2 = LOW

Why the nENABLE pull-up matters
The DRV8825's EN input has an internal pull-down: a **floating** nENABLE means **enabled**. While the XIAO boots, resets or sits in BOOTSEL its GPIOs are high-impedance, so without the pull-up the motors would be live with nothing in control. Check the strapping against TI's DRV8825 datasheet when you assemble.

4.3 · MPU6050 (GY-521)

GY-521 pin	XIAO	Notes
VCC	3V3	The breakout's own regulator is bypassed; it works fed from 3V3
GND	GND	Common ground
SDA	GPIO6	I²C1
SCL	GPIO7	I²C1
INT	GPIO5	Push-pull 3.3 V, safe for the RP2350

Mount the board **rigidly** (no foam) at axle height, near the wheel axis, with its **X axis pointing forward** and Z up. The filter computes pitch as `atan2(accel_x, accel_z)` and integrates gyro Y, so any other orientation balances against the wrong axis.

4.4 · XIAO pinout

Every header pad of the XIAO, on the edge it sits on and in the order Seeed's own KiCad footprint gives — generated by `tools/hardware/pinout.py` from `docs/reference/boards/xiao-rp2350.md`, not drawn from a photo. Blue pads carry a signal and are labelled with it. The drawing is not to scale. The GY-521 and the DRV8825 carrier are not drawn: no vendor board file for them is checked in, so read their header order off each module's own silkscreen.

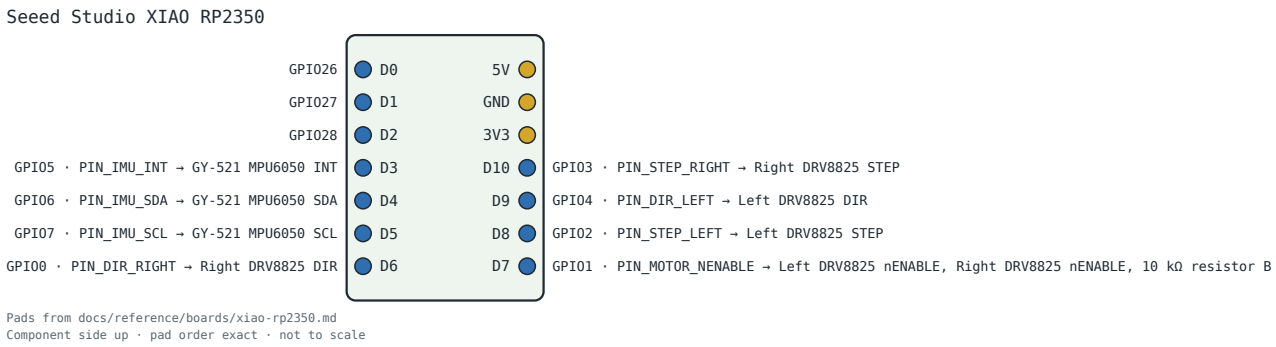


Figure 2: XIAO RP2350, component side up, USB-C at the top.

5 · Power

The **3S 18650 pack** (9.0–12.6 V, through its BMS) feeds both DRV8825 VMOT pins directly, each with a 100 µF electrolytic close to the pins, and an **LM2596 buck converter** whose output goes to the XIAO's **5V** pad. The XIAO's 3.3 V regulator then supplies the MPU6050, the DRV8825 logic straps and the nENABLE pull-up.

During bench work the XIAO can run from USB alone: the IMU, the console and telemetry all work with no motor power connected.

Golden rule

Common ground everywhere: battery −, both DRV8825 GND pins, the buck converter's GND and the XIAO's GND. USB and VMOT grounds are common.

Never power the motors from the XIAO. Not from its battery pads, not from its 5V or 3V3 pads — motor current comes from the pack through VMOT only.

6 · Assembly Steps

Build the 3S pack

Three cells in series keep VMOT between 9.0 V (empty) and 12.6 V (full), above the DRV8825's 8.2 V minimum for the whole discharge. A 2S pack is below it for most of the curve, so 3S is the only supported pack.

1. **Match the cells.** Same model; resting voltages within 0.05 V of each other. Reject any cell below 2.5 V, or one that loses voltage over a few days.
2. **Insulate the positive ends** with fish-paper rings before welding — the can is the negative terminal, right beside the cap.
3. **Spot-weld pure nickel strip** (0.15 × 8 mm): short pulses, two to four welds per tab, pull-test each joint. Never solder to a cell.
4. **Fit the 3S BMS**: B− to the pack's negative end, the B1/B2 balance taps to the two junctions between cells, B+ to the positive end; the load and charger connect to P+ / P−.
5. **Wrap and check**: Kapton or heat-shrink over the pack, then measure 9.0–12.6 V at P+ / P− before anything else is connected.

> **The BMS cutoff drops the robot**
> When the pack runs flat the BMS opens the circuit and the motors stop at once. `stat`'s battery figure reads the XIAO's own divider, not this pack, so nothing warns you first. Stop when runs get noticeably shorter, and recharge.

Wire it up

1. **Wire power first.** Connect the pack to the buck converter's input and, with nothing on its output, adjust it until a multimeter reads **5.0 V**. Only then run the 5 V feed to the XIAO's 5V pad and the common ground.
2. **Wire VMOT** from the pack to each DRV8825, with a 100 µF electrolytic across VMOT and GND MOT at each carrier, stripe (−) to ground.
3. **Fit the nENABLE pull-up**: 10 kΩ from the shared nENABLE line to 3V3, then run that line to both carriers' nENABLE and to GPIO1.
4. **Strap each DRV8825**: nSLEEP and nRESET to 3V3; M0 and M1 HIGH, M2 LOW for 1/8 microstep.
5. **Wire STEP and DIR** to GPIO2/4 (left) and GPIO3/0 (right).
6. **Wire the MPU6050**: VCC to 3V3, GND, SDA to GPIO6, SCL to GPIO7, INT to GPIO5. Mount it rigidly at axle height, X forward.
7. **Connect the stepper coils** (A1/A2, B1/B2) — but set Vref first (§6.1).
8. **Check before power-up** with a multimeter: continuity on every ground, no continuity between VMOT and 3V3, and nENABLE reading 3V3 through the pull-up with the XIAO unpowered.

> **Do not substitute internal pulls**
> On the RP2350's A2 stepping the internal pull-downs cannot hold an undriven pin low (erratum RP2350-E9: a floating input latches at ~2.2 V). Every pull in this harness is an external resistor or an actively driven line, and the firmware uses no internal pull on the IMU's INT line.

6.1 · Set the DRV8825 current limit (Vref)

Do this per driver, motors connected, before the first `arm`:

1. Power VMOT and the logic, drivers disabled (the default).
2. Measure Vref between the trim-pot wiper and GND.
3. The DRV8825 limits current to **2 × Vref**. For a typical 1.5 A/phase NEMA17 at 1/8 microstep, start conservative: Vref ≈ 0.4 V gives 0.8 A/phase. Raise it only if steps are skipped under load, with cooling as needed.

7 · Build & Flash the Firmware

Builds run inside the `pico-sdk` container — **no local toolchain is required**. The XIAO enumerates as a USB CDC serial port (`/dev/cu.usbmodem*`), which `just` auto-detects.

```
# From the repo root
just balancebot::build       # containerized cmake + ninja -> build/balancebot.uf2
just balancebot::test        # host unit tests: PID, filter, CRC, LQR, params
just balancebot::flash       # picotool: reboots a running board into BOOTSEL itself
just balancebot::flash-uf2   # or: prints the drag-and-drop steps
just balancebot::monitor     # USB CDC serial console
```

Flashing without picotool (BOOTSEL)
Hold **BOOT** on the XIAO and plug in USB (or hold BOOT and tap RESET). A drive named **RP2350** appears; copy `build/balancebot.uf2` onto it and the board reboots into the firmware.

Tuned gains live in the **second-to-last** 4 KB sector of the 2 MB flash. Not the last: on the A2 stepping a UF2 download erases the last flash block (erratum RP2350-E10), so a reflash keeps saved gains.

8 · First Boot & Calibration

Open the console with `just balancebot::monitor` and type `help`. The commands come from `src/cli.c`:

Command	Effect
`stat`	State, IMU status, angle and setpoint, step rate, loop time, battery, flash saves, stream
`get`	List the tunable parameters
`set <name> <value>`	Set a parameter: `kp ki kd vkp vki alpha maxrate`, or an LQR one (§10.2), which re-solves the gains
`ctrl [pid|lqr]`	Show or switch the balance law, live and bumpless
`stream on|off [hz]`	CSV telemetry, default 25 Hz
`arm · disarm`	Start / stop balancing
`rate <l|r> <sps>`	Open-loop motor test, disarmed only
`cal`	Gyro bias calibration — keep the robot still
`save · load`	Persist / reload parameters (flash), disarmed only

CSV columns: `t_ms,state,angle_deg,gyro_dps,setpoint_deg,rate_sps,loop_us,ctrl,pos_m`. `ctrl` is 0 for PID and 1 for LQR; `pos_m` is the commanded wheel travel since arming. New columns are only ever appended. The status LED blinks at 1 Hz in IDLE, stays on in RUN and blinks fast (5 Hz) in FAULT.

1. With **no motor power**, check that `stat` reports `imu: ok`. Then `stream on` and tilt the frame: the angle follows, and reads ≈ 0° level.
2. Run `cal` with the robot held still, and watch for drift over a minute.
3. Connect motor power with the wheels off the ground. `rate l 400` and `rate r 400` must spin both wheels in the **same** direction, the way the robot would drive; `rate l -400` reverses cleanly; `rate l 0` stops.

What the battery figure measures
`stat`'s battery figure comes from the XIAO's own sense divider on GPIO29, which watches the XIAO's battery pads — not the motor pack.

9 · Functional Checkout

	Check	Expected result
□	Pull-up	nENABLE reads 3V3 with the XIAO unpowered or in BOOTSEL
□	Buck output	5.0 V at the XIAO 5V pad, set before the XIAO is connected
□	Console	`help` lists the commands; LED blinks at 1 Hz (IDLE)
□	IMU	`stat` shows `imu: ok`; streamed angle tracks a tilt, ≈ 0° level
□	Gyro cal	After `cal`, no visible drift over a minute
□	Vref	Set on both drivers before the first `arm`
□	Motors	`rate l 400` / `rate r 400`: both wheels turn the same way; `-400` reverses
□	Tilt cutoff	Tipped past 45° the drivers drop out and the LED blinks fast
□	Arm	`arm`, or held upright 3 s, enters RUN with the LED steady

10 · Tuning: PID and LQR

Two balance laws share the same path to the wheels. Both output a wheel **acceleration**, which the loop integrates into a step rate. `ctrl pid` and `ctrl lqr` switch between them while the robot balances. Tune the PID first: the LQR takes its sense of "falling forward" from the same `IMU_PITCH_SIGN`, so a robot that balances on the PID has the LQR's sign right too.

10.1 · PID

The angle loop starts from `kp` 28, `ki` 0, `kd` 1.2; the velocity loop from `vkp` 0.004, `vki` 0.0005 (defaults in `src/config.h`). Tune tethered over USB:

1. Bench-check with `rate l 800` / `rate r 800`: both wheels spin smoothly, and `rate l 0` stops.
2. Set `ki`, `vkp` and `vki` to 0. Raise `kp` until the robot visibly fights the fall; fast oscillation means it is too high.
3. Raise `kd` to damp the oscillation.
4. Restore small `vkp` / `vki` so it holds station instead of wandering.
5. `save` to persist the gains (disarmed only; `stat` reports whether it worked).

10.2 · LQR

The LQR uses a model of the robot as an inverted pendulum on wheels. It needs one measured length and the wheel size; the four gains are then computed, not tuned. The XIAO solves them on core 0 at boot and every time one of these parameters changes:

Param	Meaning	Default
`l_eff_m`	Equivalent pendulum length, from the swing test below (m)	0.12
`wheel_diam_m`	Wheel diameter (m)	0.09
`steps_per_rev`	Microsteps per wheel revolution	1600
`q_theta q_dtheta`	How hard to fight lean angle and lean rate	400, 1
`q_x q_dx`	How hard to hold position and stop drifting	25, 4
`r`	Cost of wheel acceleration; raise it for gentler motion	0.04

The `l_eff_m` and `wheel_diam_m` defaults are placeholders. Measure both before trusting the LQR.

The swing test for `l_eff_m`:

1. Disarm and disconnect motor power.
2. Hang the robot from its wheel axle so the body swings freely below it, like a pendulum. A rod through both wheel hubs, resting on two supports, works.
3. Pull the body a few degrees aside and let go. Small swings matter: keep it under about 10°.
4. Time 10 full swings (out and back) with a stopwatch and divide by 10. That is the period T in seconds.
5. Compute $L_{\text{eff}} = g(T/2\pi)^2$ with $g = 9.81\text{m/s}^2$. For example, $T = 0.70$ s gives $L_{\text{eff}} \approx 0.122$ m.

Hanging below the axle it swings like the upright robot falls. Both depend on the same mass distribution, so this one number replaces the guesswork of locating the centre of mass and the moment of inertia.

Then, on the console:

1. `set l_eff_m` and `set wheel_diam_m` to the measured values. Each change prints the new `K`, the closed-loop `rho` (it must be below 1) and the solver's `residual`. A change whose solve fails is rejected with the reason, and the old value and gains stay.
2. Balance on the PID, then `ctrl lqr`. The robot holds the spot it switched at instead of wandering back to where it armed.
3. Watch `pos_m` and `angle_deg` in the stream. If it rocks, raise `r` or lower `q_theta`. If it drifts away, raise `q_x`.
4. `ctrl pid` switches back. That switch is smooth only with a nonzero `ki`; with `ki 0` the wheels may jerk once.
5. `save` keeps the mode and the model. The gains are not saved but re-solved at every boot. If that solve fails, the robot runs the PID and `stat` says why.

> **ctrl lqr refused**
> `ctrl lqr` is refused while the XIAO holds no valid gains, for example after a `load` whose model cannot be solved. The message names the reason. Fix the parameter, and the next successful `set` makes LQR available again.

11 · Troubleshooting

Symptom	Likely cause & fix
Motors energise while the XIAO boots or sits in BOOTSEL	The nENABLE pull-up is missing or not on 3V3. The DRV8825's internal pull-down enables a floating EN input.
Robot drives **into** the fall on first arm	Pitch sign reversed. Flip `IMU_PITCH_SIGN` in `src/config.h`, or rotate the sensor 180°.
Wheels turn opposite ways on `rate l` / `rate r`	One motor's coil pair is reversed. Swap one coil pair on that motor; firmware already inverts the right DIR line.
Skipped steps or loss of balance under load	Current limit too low. Raise Vref gradually (limit = 2 × Vref) and cool the drivers.
`stat` shows `imu: NOT RESPONDING`	No I²C answer from the MPU6050. Check SDA on GPIO6, SCL on GPIO7, VCC on 3V3 and ground. Without the IMU the hold-upright gesture never arms, and `arm` drops straight to FAULT.
Drops out to FAULT mid-balance	Pitch passed the 45° tilt cutoff. It clears after 2 s held upright within 5°.
An undriven input reads HIGH (~2.2 V)	RP2350-E9 on the A2 stepping: internal pull-downs do not hold. Use an external resistor or a driven line.
`ctrl lqr` answers `refused`	No valid LQR gains: the last solve failed (the message says why). `q_x 0` is refused on purpose — nothing would hold position. Correct the parameter with `set`.
Robot jerks when switching back to `ctrl pid`	With `ki 0` the PID has no integral to absorb the difference. Set a small `ki`, or switch while the robot is calm.

Symptom	Likely cause & fix
Saved gains lost after a power cycle	`save` refuses while armed; `disarm` first and check `stat`'s `saves` line.
Board will not flash	Hold BOOT while plugging in USB (or hold BOOT and tap RESET) and copy the UF2 by hand.

Authoritative pin data: `packages/robotics/balancebot/src/pin_config.h` · Design: `docs/requirements/PRD-012-balancebot.md`, `docs/decisions/ADR-018-pico-sdk-balancebot.md`, `docs/decisions/ADR-026-balancebot-lqr-live-switch.md`
Regenerate this PDF with `just balancebot::build-guide`.